UNIVERSITÉ CHEIKH ANTA DIOP
DE DAKAR
LUX MEA LEX

Faculté des Sciences et Techniques
Département d'Informatique

PPT 2 / 4

Systèmes distribués & Cloud

Comprendre les services, leurs communications et leurs environnements de déploiement



- Services
- Communications
- Déploiement
- Scalabilité

- Sécurité
- Disponibilité
- Performance
- Cloud computing

Prof. Issa LY
UCAD – Département d'Informatique

Université Cheikh Anta Diop de Dakar | Faculté des Sciences et Techniques

LUX MEA LEX

PARTIE 1

Systèmes distribués

Passer d'une application centrale à plusieurs services



- Modularité
- Scalabilité
- Résilience
- Cloud-ready

Comprendre le principe avant les outils

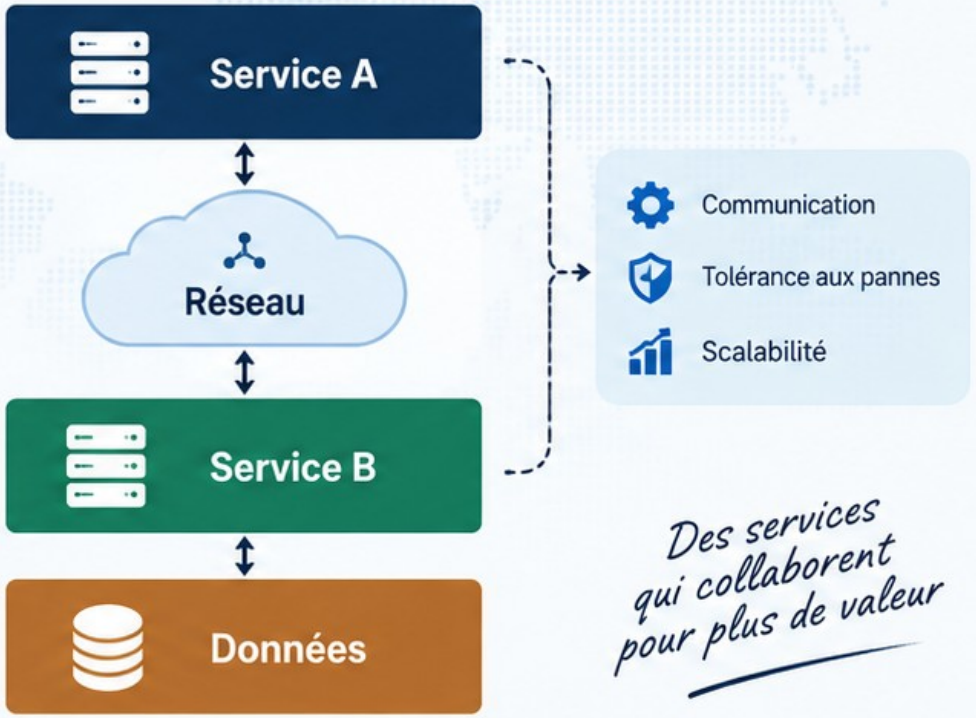
Prof. Issa LY | UCAD – Département d'Informatique

Prof. Issa LY
Département d'Informatique
Faculté des Sciences et Techniques

DÉFINITION

Principe d'un système distribué

- Un système distribué est composé de plusieurs composants autonomes qui coopèrent par le réseau.
- Chaque composant peut être déployé séparément.
- Le réseau devient une partie active de l'architecture.
- Les pannes partielles doivent être prévues dès la conception.



À retenir

Modularité
Composants autonomes et déployables séparément.

Résilience
Le système continue de fonctionner malgré des pannes partielles.

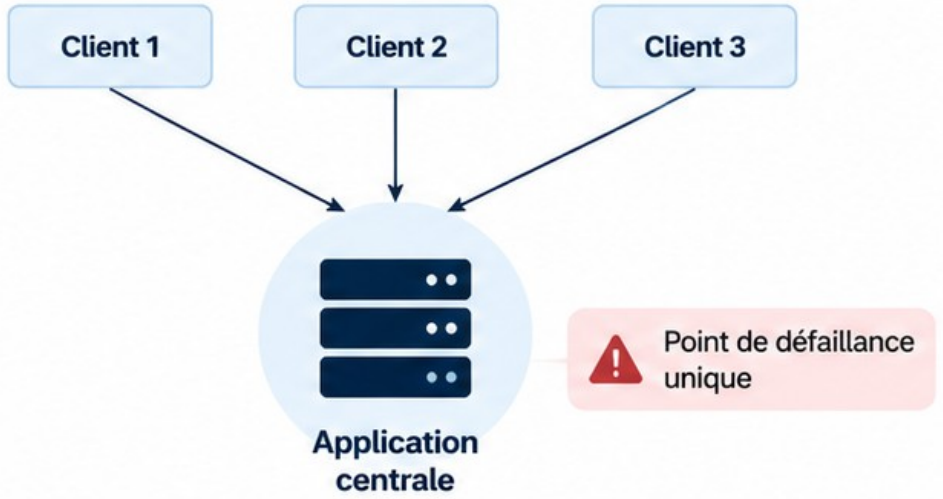
Scalabilité
Ajout de nouveaux services selon les besoins.

COMPARER

Centralisé ou distribué ?

Centralisé

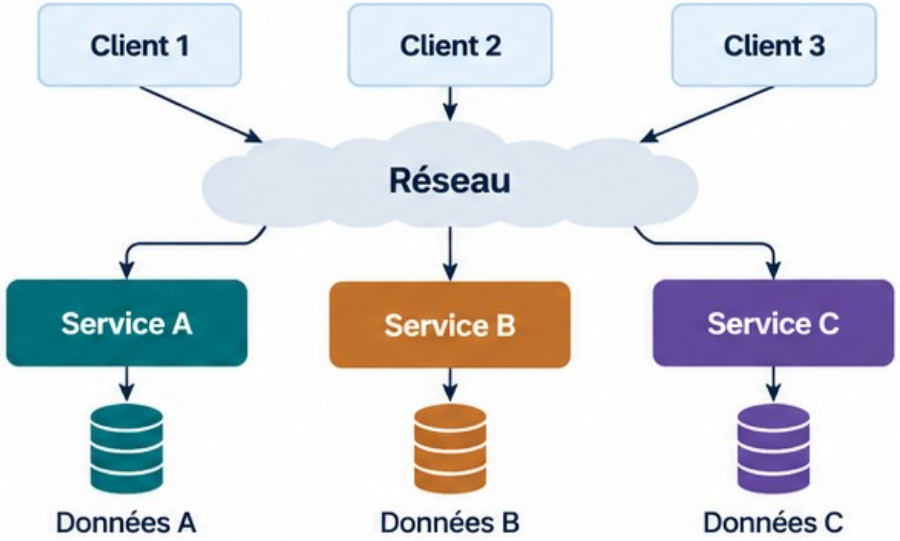
- Un ensemble principal
- Flux internes simples
- Déploiement plus direct
- Panne potentiellement globale



VS

Distribué

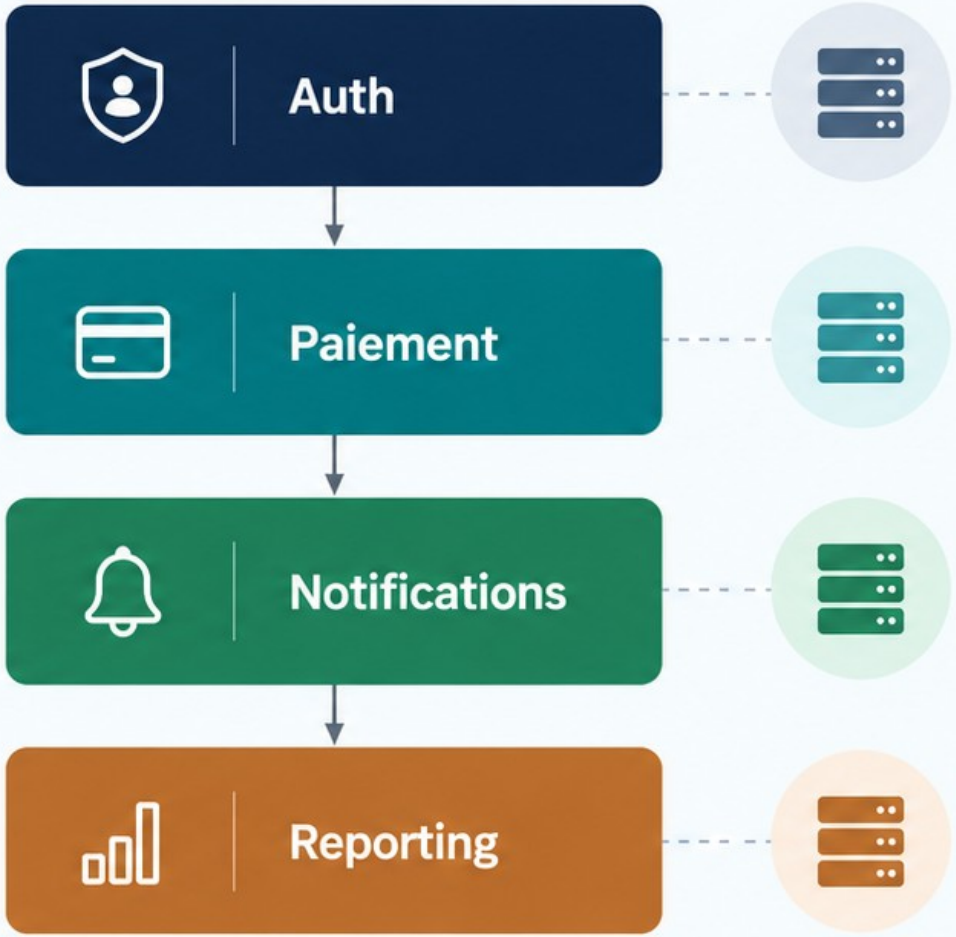
- Plusieurs services
- Flux réseau nombreux
- Évolution indépendante
- Pannes partielles à gérer



À retenir : Le choix dépend des besoins en scalabilité, disponibilité et indépendance des services.

Pourquoi distribuer ?

- Certaines fonctions n’ont pas les mêmes contraintes de charge.
- Plusieurs équipes peuvent travailler séparément.
- Certains domaines doivent être isolés pour des raisons de sécurité.
- La montée en charge peut cibler un service précis.

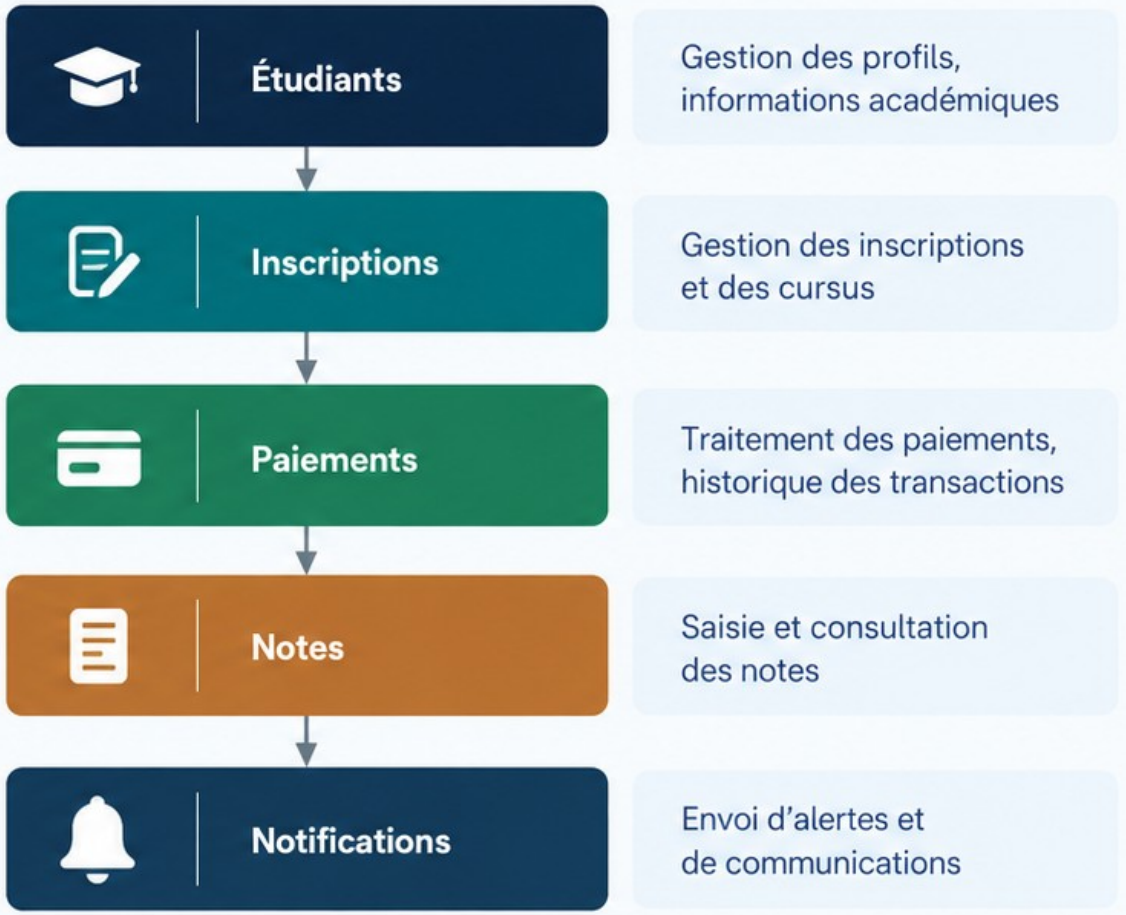


CONCEPTION

Découper par domaine métier

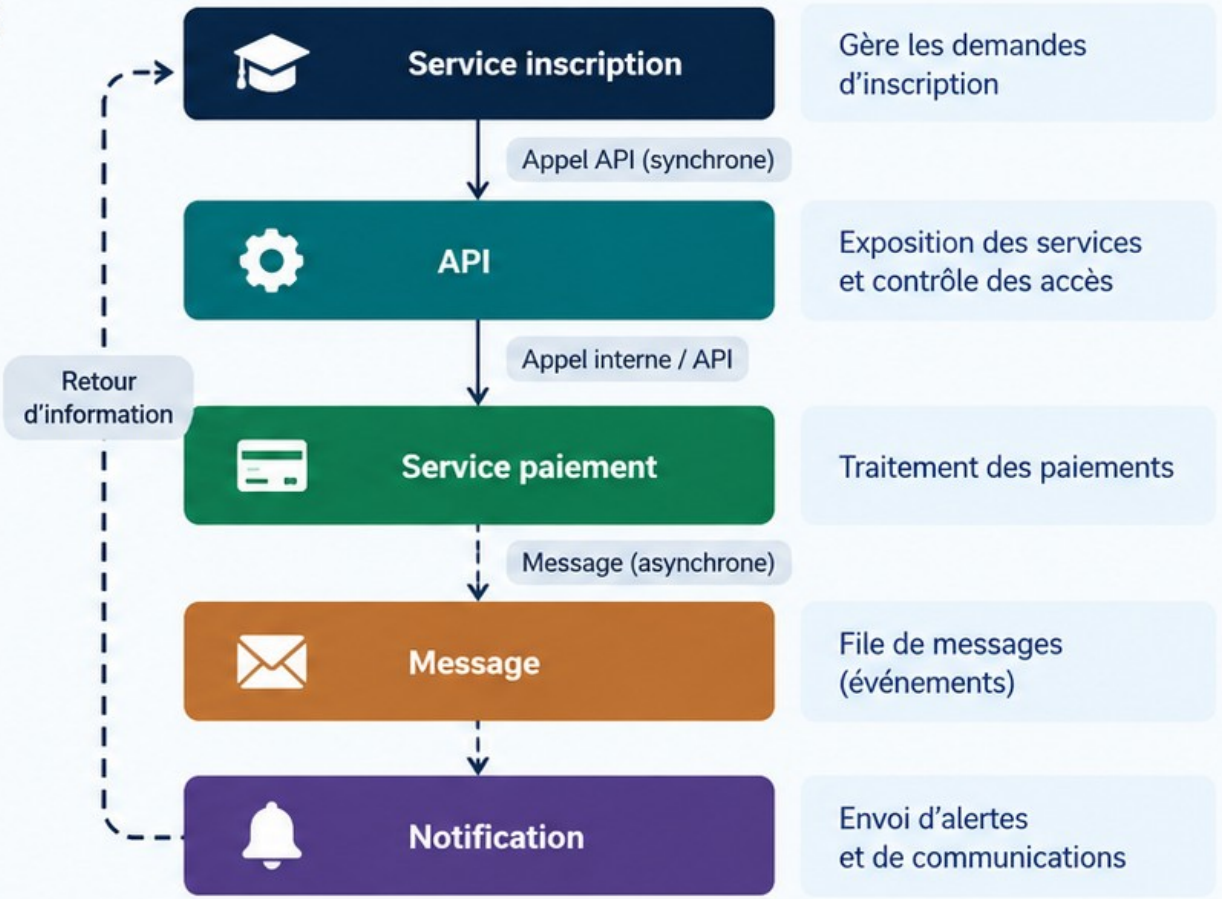
- Un service doit correspondre à une responsabilité cohérente.
- Le découpage technique ne doit pas suivre uniquement les tables.
- Chaque service doit avoir des frontières claires.
- Un mauvais découpage crée plus de complexité que de valeur.

À retenir : Un bon découpage suit les capacités métier, les besoins d'indépendance et les règles de cohérence, pas seulement la structure des données.



Communication entre services

- Les services communiquent par API, messages ou appels internes.
- La communication peut être synchrone ou asynchrone.
- Chaque échange doit être observable et sécurisé.
- Le nombre de flux augmente rapidement avec les services.



À retenir : Bien concevoir les flux dès le début permet un système plus simple, plus fiable et plus évolutif.

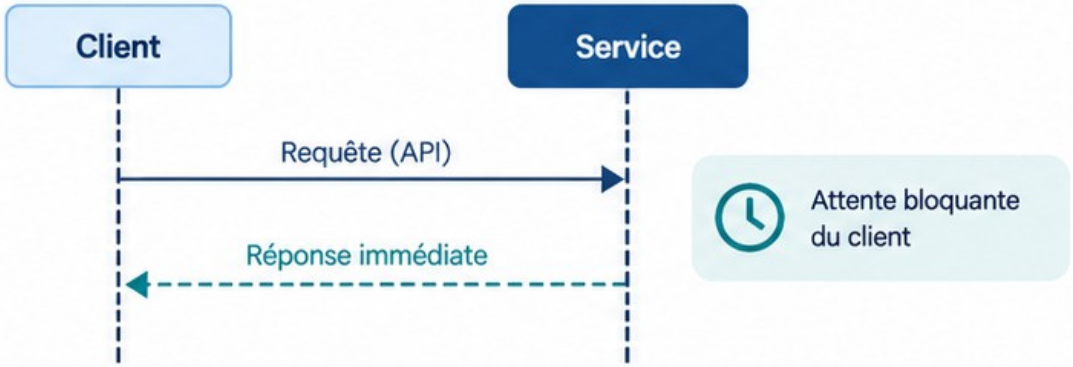
COMPARER

Synchrone ou asynchrone ?

Synchrone

Réponse immédiate

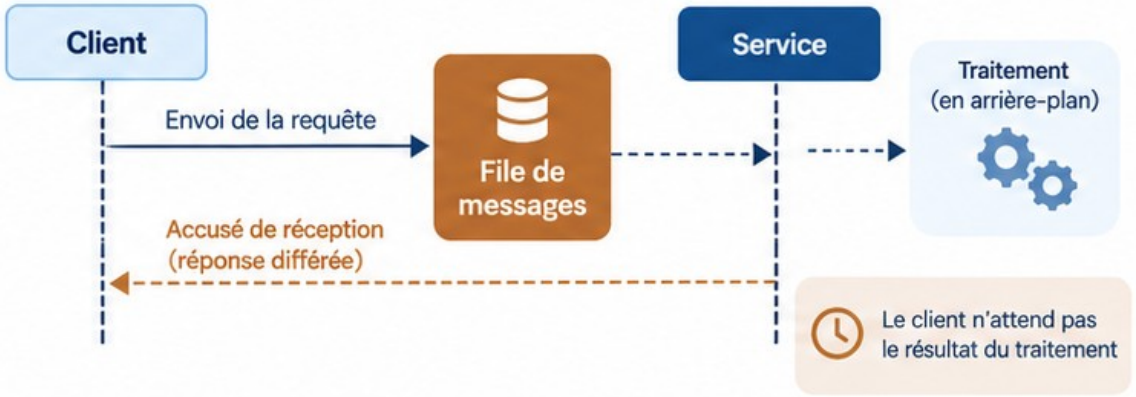
- Le client attend la réponse.
- Simple à comprendre.
- Adapté aux consultations ou validations immédiates.
- Sensible aux délais et indisponibilités.



Asynchrone

Traitement en arrière-plan

- Le traitement peut continuer plus tard.
- Découple les services.
- Adapté aux notifications et traitements longs.
- Demande une file et une gestion des erreurs.



À retenir : Le synchrone est adapté aux réponses rapides, l'asynchrone aux traitements longs et à la scalabilité.

REST dans les systèmes distribués

- REST expose des ressources via HTTP.
- Les méthodes indiquent l'action attendue.
- Le format JSON facilite l'échange entre services.
- REST reste simple mais nécessite sécurité, versioning et supervision.

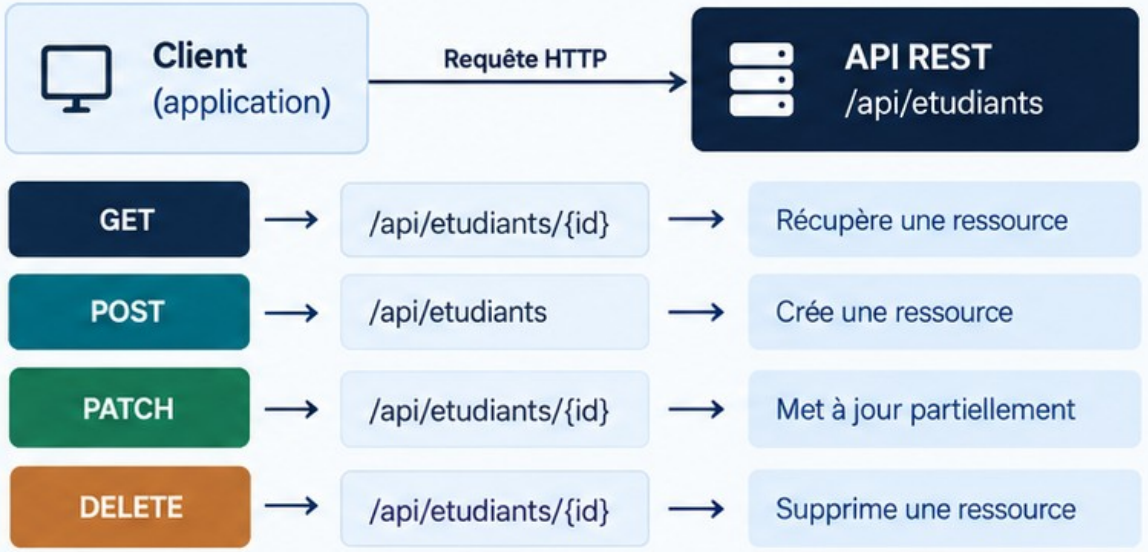
Principes REST

Ressources identifiées par une URI

Méthodes HTTP indiquent l'action

JSON format standard d'échange

Sécurité authentification, autorisation, supervision



Exemple de réponse JSON

HTTP/1.1 200 OK

```
{
  "id": 123,
  "nom": "Diallo",
  "prenom": "Awa",
  "email": "awa.diallo@ucad.edu.sn",
  "filiere": "Informatique"
}
```

À retenir : REST est simple et efficace, mais doit être bien sécurisé, versionné et supervisé dans un système distribué.

SYNTHÈSE

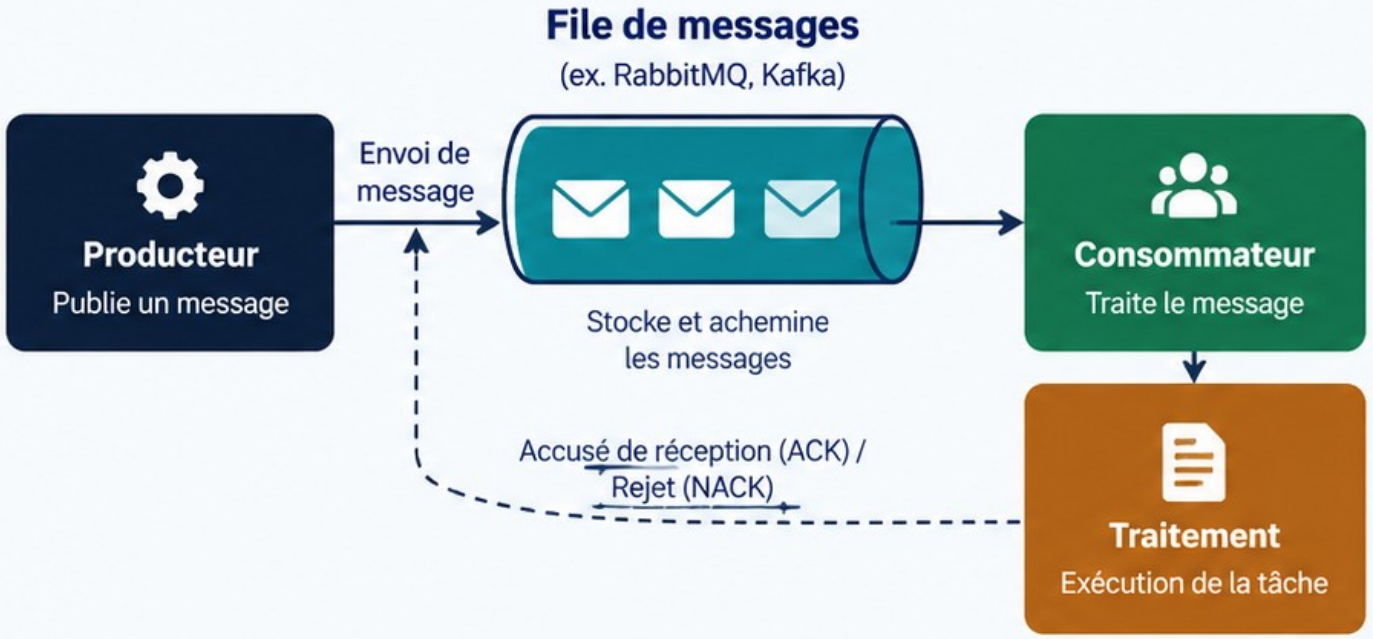
Exemple REST universitaire

Service	Endpoint	Rôle
Étudiants	GET /api/etudiants/125	Lire un dossier étudiant
Inscriptions	POST /api/inscriptions	Créer une inscription
Paiements	POST /api/paiements	Enregistrer un paiement
Notifications	POST /api/messages	Envoyer une notification

À retenir : Chaque service expose des endpoints clairs et joue un rôle métier précis.

Messaging

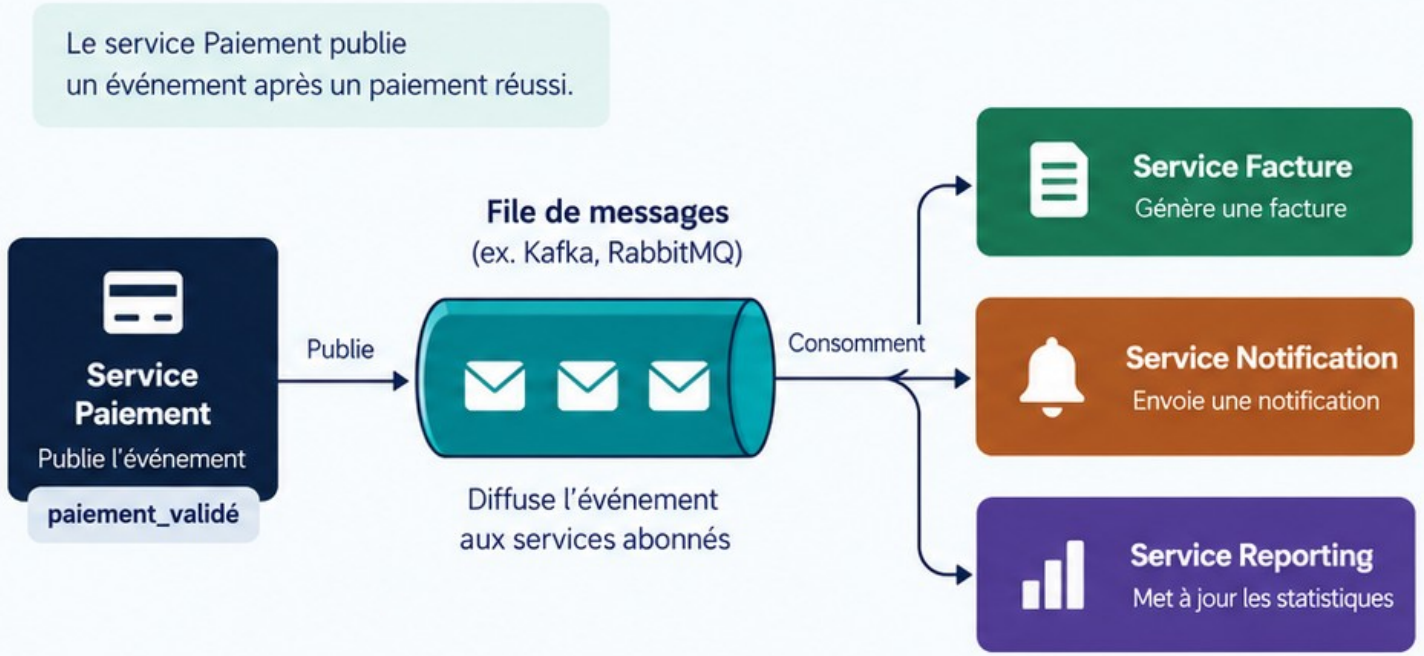
- Le messaging transmet un événement ou une commande à traiter plus tard.
- Il limite les dépendances directes entre services.
- Il absorbe les pics et améliore la résilience.
- Il demande une gestion des accusés de réception et des rejets.



À retenir : Le messaging découple les services, permet un traitement asynchrone et améliore la résilience du système, mais nécessite une gestion des accusés et des erreurs.

Événement métier

- Un événement décrit quelque chose qui s'est produit.
- Exemple : paiement_validé, inscription_confirmée, note_publiée.
- Les autres services réagissent sans bloquer le service source.
- Le système devient plus flexible mais plus difficile à suivre.



À retenir : Les événements métier permettent de découpler les services et d'améliorer la flexibilité du système, mais nécessitent une bonne gestion des messages et de la traçabilité.

COMPARER

REST ou messaging ?

REST

Réponse immédiate

- Demande/réponse immédiate.
- Facile à tester.
- Bon pour lecture et validation.
- Dépendance temporelle entre services.



Messaging

Traitement différé

- Traitement différé.
- Découplage fort.
- Bon pour notifications et tâches lourdes.
- Plus complexe à superviser.



À retenir : Utiliser REST pour les échanges simples et immédiats, et le messaging pour les traitements asynchrones et le découplage des services.

PARTIE 2

Cloud

Déployer et faire évoluer l’infrastructure



Comprendre le principe avant les outils

Pourquoi le Cloud ?

- Le Cloud fournit des ressources **à la demande**.
- Il peut **accélérer le déploiement** et la **montée en charge**.
- Il déplace certaines **responsabilités** vers le fournisseur.
- Le choix Cloud dépend du **besoin**, du **coût**, de la **conformité** et de **l'exploitation**.



Besoin
Identifier les exigences

Ressource
Allouer à la demande

Déploiement
Accélérer la mise en service

Exploitation
Superviser et optimiser

À retenir : Le Cloud permet de s'adapter rapidement aux besoins, mais nécessite une analyse du coût, de la sécurité, de la conformité et de l'exploitation.

COMPARER

On-premise ou Cloud ?

Deux approches pour les mêmes objectifs, des choix différents selon le contexte.

Il n'y a pas de bon ou mauvais choix, mais une solution adaptée à vos besoins.

On-premise

Votre infrastructure, vos locaux

- **Contrôle direct** du matériel
- Dépend des **équipes internes**
- **Investissement initial** plus élevé
- Très adapté à certaines **contraintes locales**



Infrastructure sur site sous votre contrôle

Cloud

Des ressources à la demande

- Provisionnement **rapide**
- Services **managés** disponibles
- Facturation **à l'usage**
- Dépendance au **fournisseur** et au **réseau**



Infrastructure dans le cloud accessible partout

À retenir : Choisissez en fonction de vos besoins métier, de votre budget, de vos contraintes réglementaires et de vos compétences internes.

SYNTHÈSE

IaaS, PaaS, SaaS

Trois modèles pour des niveaux de responsabilité différents

Même objectif : fournir une valeur métier plus rapidement, avec moins de contraintes.

Plus de valeur métier

Plus de contrôle



Software as a Service

Une application prête à utiliser

- Application prête à l'emploi
- Configuration limitée
- Aucune gestion de l'infrastructure
- Accès via un navigateur

Exemples :

Google Workspace · Microsoft 365 · Salesforce



Platform as a Service

Une plateforme pour développer et exécuter vos applications

- Environnement d'exécution géré
- Outils de développement intégrés
- Gestion de l'infrastructure par le fournisseur
- Focus sur le code et la logique métier

Exemples :

Heroku · Azure App Service · Google App Engine



Infrastructure as a Service

Des ressources infrastructureles à la demande

- VM, réseau, stockage
- Contrôle complet de l'environnement
- Gestion de l'OS, des applications, des données
- Très flexible et scalable

Exemples :

AWS · Microsoft Azure · Google Cloud

Qui gère quoi ?

SaaS
Le fournisseur gère tout

PaaS
Le fournisseur gère l'infrastructure
Vous gérez l'application

IaaS
Vous gérez l'OS, les applications et les données
Le fournisseur gère l'infrastructure physique

À retenir :

Plus vous montez dans la pile (IaaS → PaaS → SaaS), moins vous gérez l'infrastructure, mais plus vous vous concentrez sur la valeur métier.

COMPARER

Cloud public, privé et hybride

Trois approches, un même objectif : répondre au bon besoin

Cloud public

Des ressources chez un fournisseur externe

- Ressources chez un fournisseur externe
- Rapide et riche en services
- Aucun investissement initial important
- Idéal pour les charges variables



Parfait pour innover rapidement et se concentrer sur le métier.

Cloud privé

Une infrastructure dédiée

- Infrastructure dédiée à l'organisation
- Contrôle renforcé et personnalisation
- Adapté aux exigences de sécurité et de conformité
- Investissement initial plus élevé



Dans vos locaux ou chez un hébergeur dédié

Idéal pour les données sensibles et les environnements réglementés.

Cloud hybride

Le meilleur des deux mondes

- Combine le local et le Cloud
- Permet des migrations progressives
- Garde certaines données sensibles localement
- Demande une architecture réseau solide



Une approche flexible pour s'adapter à des besoins métiers variés.

À retenir : Le choix entre public, privé ou hybride dépend de vos besoins métier, de vos contraintes de sécurité, de budget et de votre stratégie IT.

Scalabilité

Adapter les ressources à la demande, sans interruption

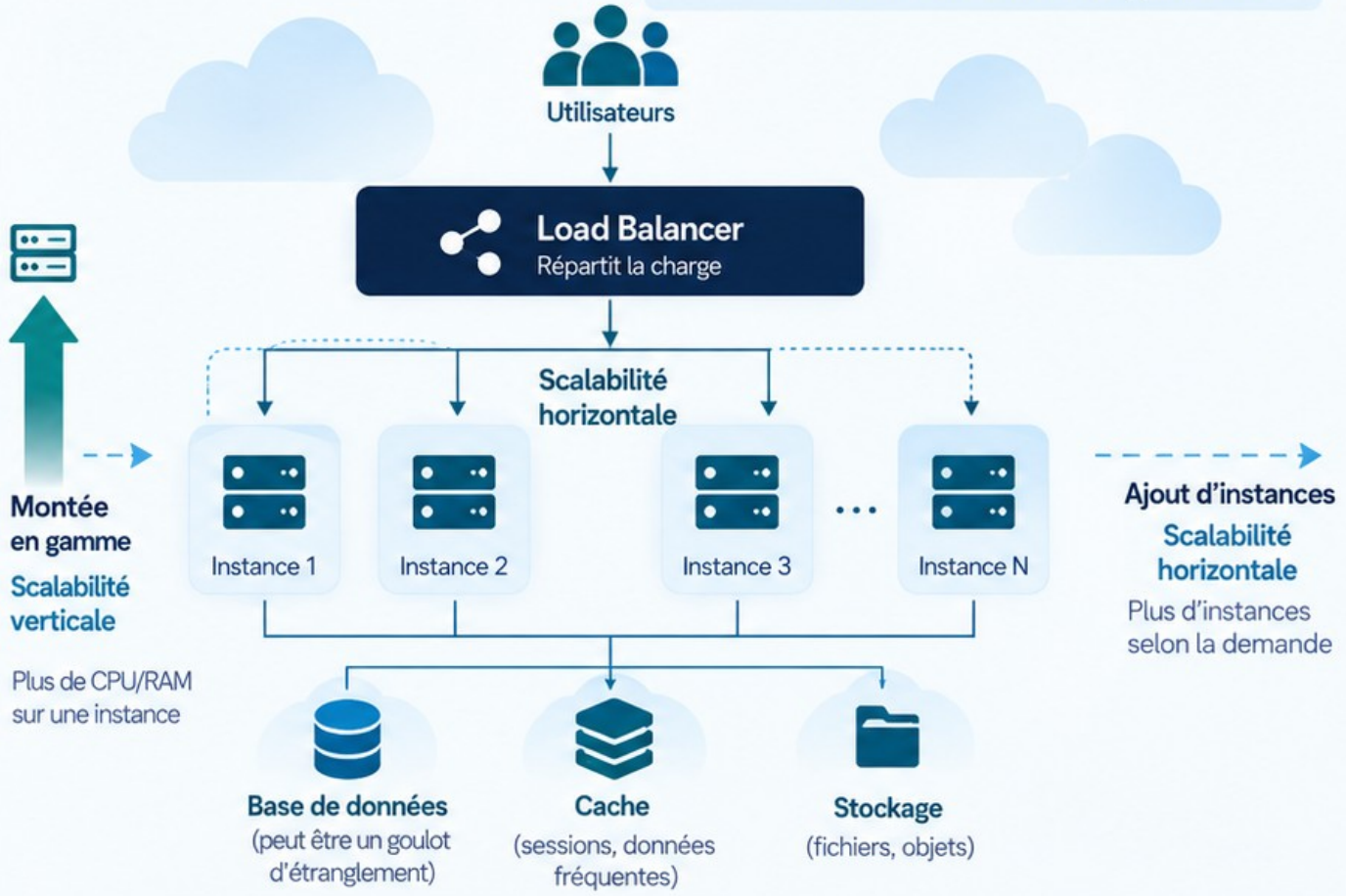
La scalabilité permet d'absorber les pics de charge et d'accompagner la croissance de votre application.

Scalabilité verticale
Ajouter des ressources (CPU, RAM) à une machine existante.

Scalabilité horizontale
Ajouter des instances pour répartir la charge.

Le Cloud facilite l'horizontalité
L'infrastructure s'adapte automatiquement, mais l'application doit être conçue pour cela.

Attention aux limites
La base de données, le cache et les sessions peuvent devenir des goulots d'étranglement.



À retenir : La scalabilité garantit la disponibilité et les performances de vos services, à condition d'anticiper les composants qui ne scalent pas automatiquement.

COMPARER

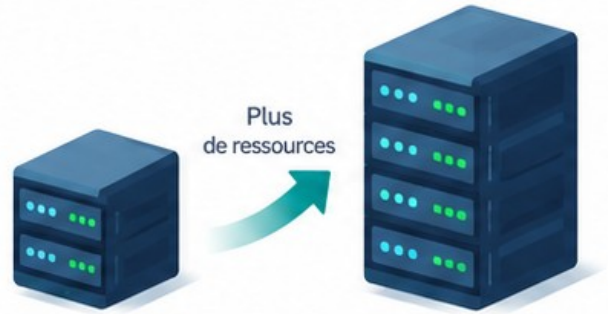
Scalabilité verticale et horizontale

Deux approches complémentaires pour répondre à la croissance de la charge

Scalabilité verticale

Ajouter des ressources à une machine existante

- Augmenter **CPU / RAM / disque**
- Simple au départ
- Limite matérielle atteinte un jour
- Risque de point unique de défaillance

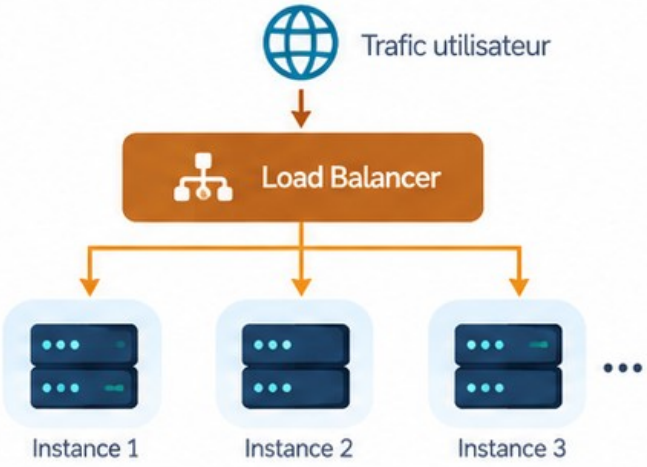


Même machine, plus puissante

Scalabilité horizontale

Ajouter plusieurs instances pour répartir la charge

- Ajouter plusieurs **instances**
- Meilleure **disponibilité**
- Nécessite un **load balancing**
- Demande sessions et données maîtrisées



Plusieurs instances, une charge **répartie**

À retenir : La scalabilité verticale est simple mais limitée.
La scalabilité horizontale est plus flexible et résiliente, mais nécessite une bonne architecture (load balancing, gestion des sessions et des données).

CONCEPTION

Haute disponibilité Cloud

Assurer la continuité de service, même en cas de panne

Redondance

Plusieurs instances ou zones de disponibilité pour éviter le point unique de défaillance.

Résilience

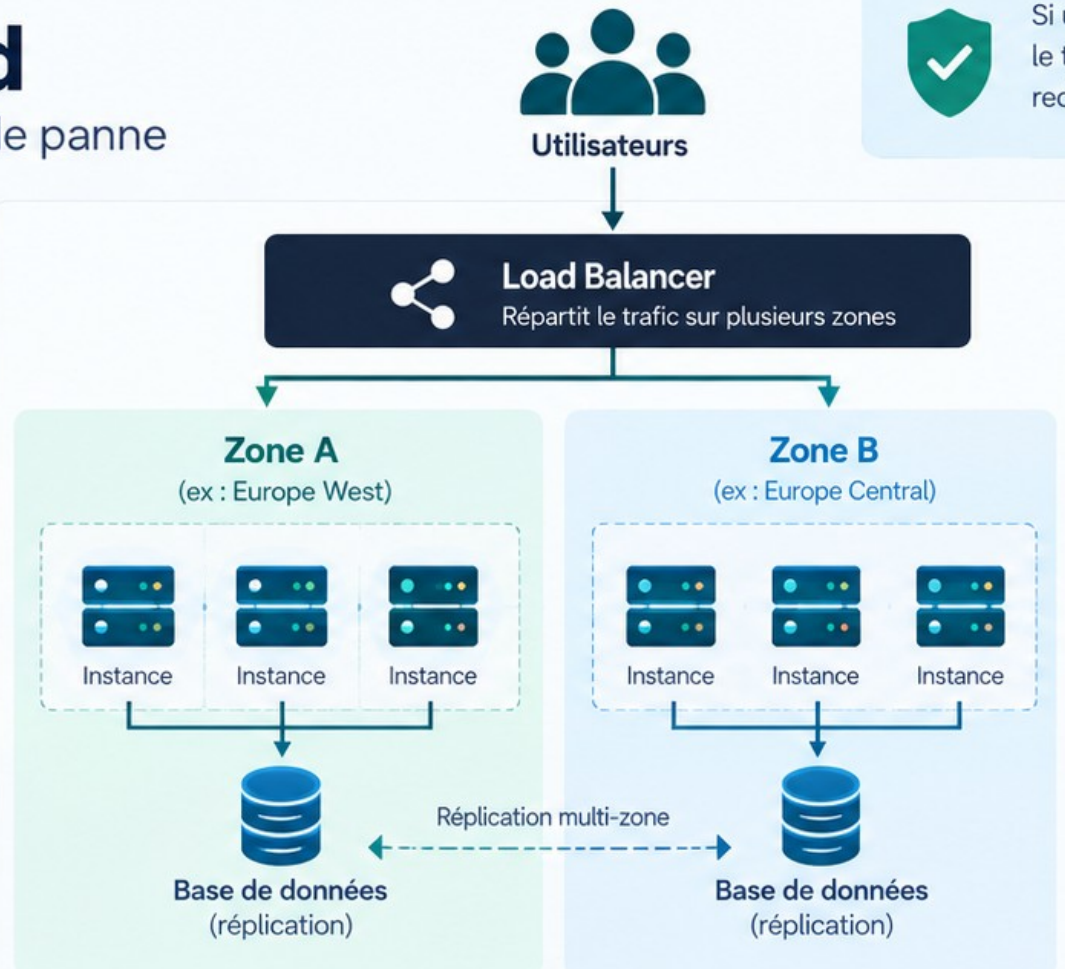
Le service doit continuer à fonctionner même en cas de perte d'un composant.

Sauvegardes

Les sauvegardes (snapshots, réplication) restent indispensables.

Coût

La haute disponibilité a un coût technique et financier (ressources, réseau, stockage, licences...).



Si une zone tombe en panne, le trafic est automatiquement redirigé vers l'autre zone.

Panne d'une zone ou d'une instance

- Détection automatique
- Basculement du trafic
- Service toujours disponible

À retenir : La haute disponibilité permet de garantir la continuité de service grâce à la redondance, mais elle augmente la complexité et les coûts.

PORTABILITÉ

Conteneurs dans le Cloud

Des applications légères, portables et faciles à déployer

"Build once, run anywhere"
La même image, du poste de dev au cloud de production.

Isolation et portabilité

Les conteneurs empaquettent l'application et ses dépendances dans une image légère.

Cycle de vie simplifié

Ils facilitent le développement, le test et la mise en production.

Exécution flexible

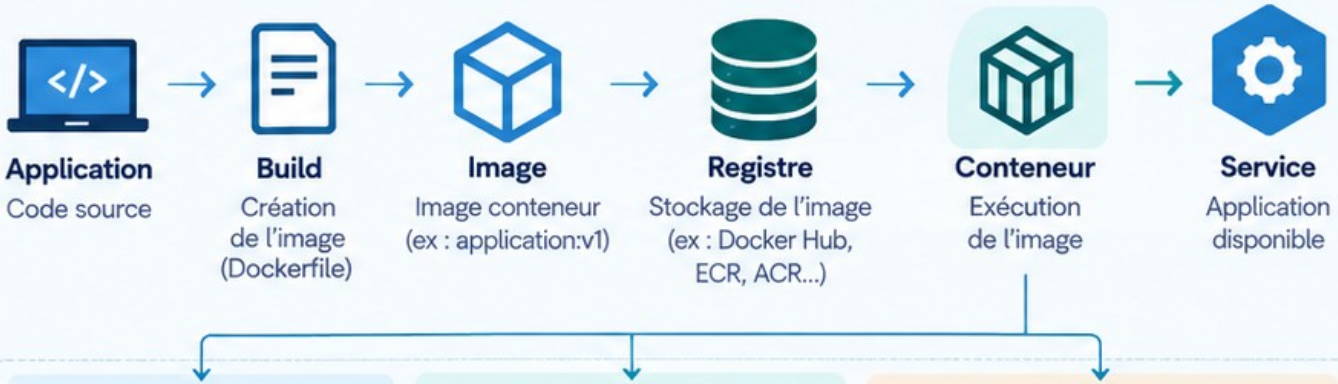
Peuvent être exécutés sur une VM, un serveur physique ou un service managé (ex. Kubernetes).

Orchestration

Devient essentielle lorsque les conteneurs se multiplient (gestion, mise à l'échelle, auto-réparation).

De l'image au service

Le cycle de vie d'un conteneur



Avantages

- Légers et rapides à démarrer
- Portables entre environnements
- Meilleure utilisation des ressources
- Facilite l'intégration et le déploiement continu

Outils d'orchestration

Kubernetes · Docker Swarm · AWS ECS · Azure AKS · Google GKE

COMPARER

VM et conteneurs dans le Cloud

Deux approches pour exécuter vos applications

Même objectif :
exécuter des applications dans le Cloud, mais avec des approches différentes.

Machine virtuelle (VM)

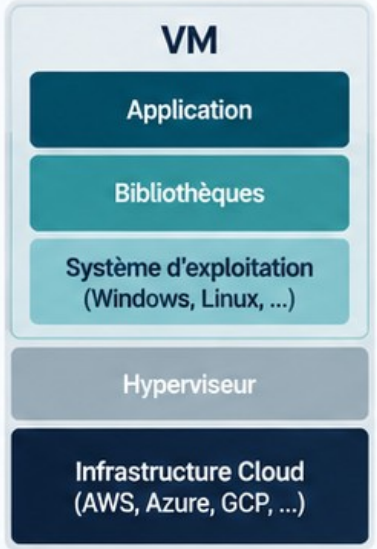
Un système complet dans le Cloud

Système d'exploitation complet
Inclut l'OS, les bibliothèques et les dépendances.

Isolation forte
Environnement totalement séparé.

Adaptée aux charges diverses
Peut exécuter différents types d'applications.

Plus lourde à démarrer
Temps de démarrage plus long
et consommation de ressources plus élevée.



Conteneur

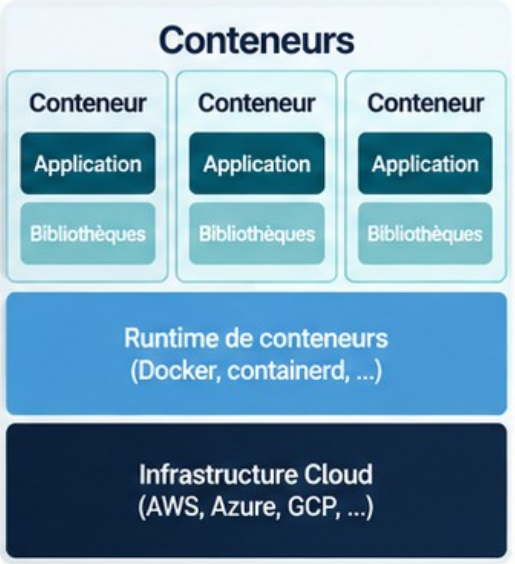
Une application légère et portable

Partage le noyau de l'hôte
Pas de système d'exploitation complet.

Léger et reproductible
Démarrage rapide et image portable.

Adapté au déploiement applicatif
Idéal pour les architectures microservices.

Dépend d'un runtime
Nécessite un moteur de conteneurs et une bonne gestion des volumes de données.



À retenir : Les VM offrent plus d'isolation et de flexibilité, les conteneurs sont plus légers, plus rapides et mieux adaptés au Cloud moderne.

Kubernetes : quand devient-il utile ?

Orchestrer des applications conteneurisées à grande échelle

Kubernetes n'est pas obligatoire pour tous les projets, mais il devient essentiel quand votre application grandit et se complexifie.

Orchestre de nombreux conteneurs

Kubernetes automatise le déploiement, la mise à l'échelle et la gestion de vos applications conteneurisées.

Gère l'ensemble du cycle de vie

Déploiement, scaling, redémarrage automatique, mise à jour sans interruption, gestion des services.

Apporte de la puissance et de la flexibilité

Adapté aux applications distribuées et aux architectures microservices.

À utiliser quand la complexité augmente

Pas nécessaire pour une petite application.
Kubernetes devient pertinent lorsque vous avez plusieurs services, des besoins de haute disponibilité et de scalabilité.



À retenir : Kubernetes apporte une grande valeur pour les applications conteneurisées distribuées, mais il doit être choisi en fonction des besoins réels et de la complexité du projet.

SYNTHÈSE

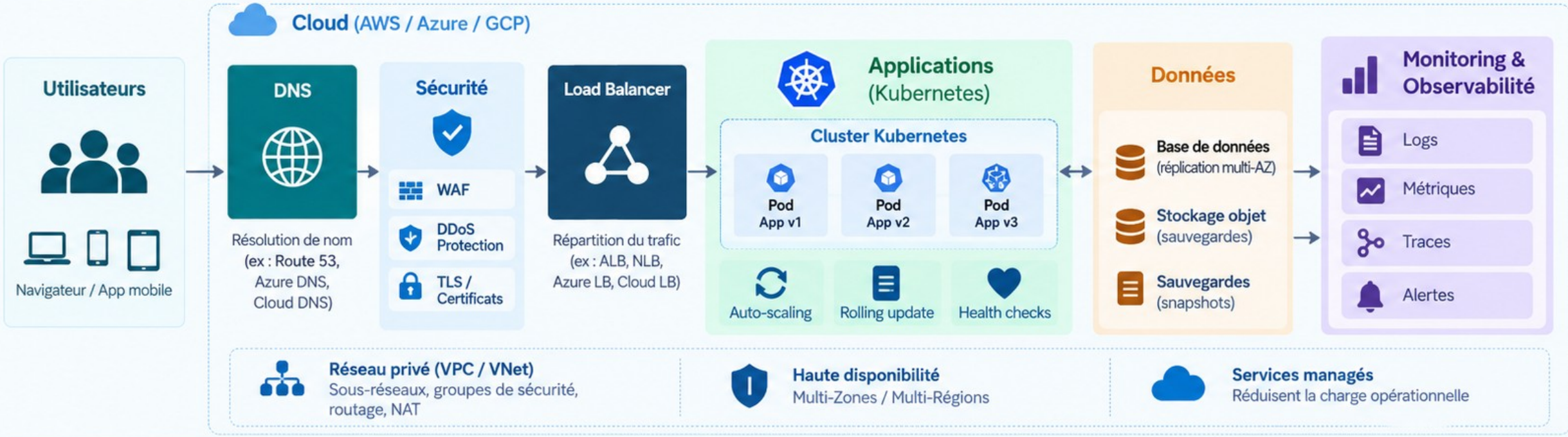
Architecture Cloud réaliste

Une architecture complète, sécurisée et scalable

- Les utilisateurs atteignent un point d'entrée sécurisé.
- Le load balancer répartit vers plusieurs instances applicatives.
- Les données sont répliquées et sauvegardées.
- La supervision mesure disponibilité, latence et erreurs.

Objectifs

- Haute disponibilité
- Scalabilité
- Sécurité
- Supervision et observabilité
- Coûts maîtrisés



À retenir : Cette architecture combine sécurité, scalabilité et résilience, en s'appuyant sur des services managés du Cloud.

ÉTUDE DE CAS

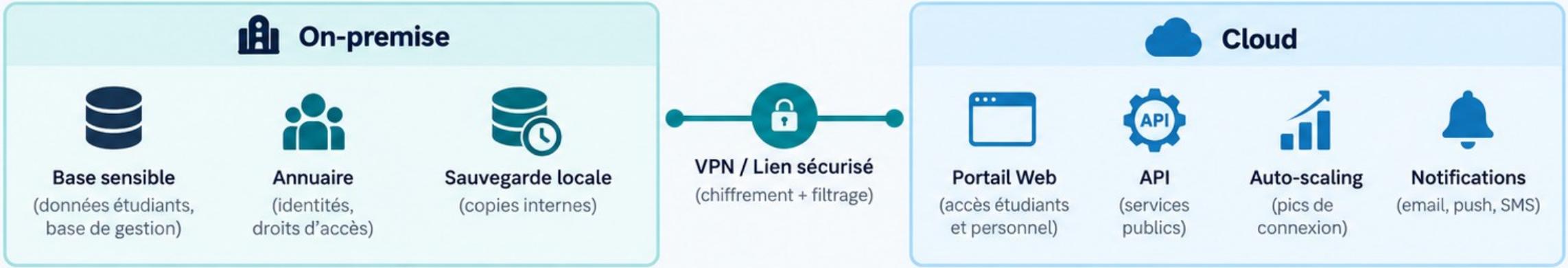
Cas : plateforme hybride

Une école conserve les données sensibles des étudiants et la base de gestion sur site, mais déploie le portail, les API publiques et certains services applicatifs dans le Cloud pour absorber les pics de connexion et améliorer la disponibilité.

- Données critiques et identité conservées localement
- Portail Web, notifications et montée en charge dans le Cloud
- Lien sécurisé entre le site et l'environnement Cloud

Décisions à justifier

1. Définir ce qui reste local
   (base sensible, annuaire, sauvegardes internes)
2. Définir ce qui passe dans le Cloud
   (frontaux, services exposés, élasticité)
3. Sécuriser les flux hybrides
   (VPN, chiffrement, filtrage)
4. Prévoir supervision, sauvegarde et plan de continuité



SYNTHÈSE

Décisions Cloud à justifier

Les bonnes questions pour faire les bons choix

Il n'existe pas de bonne réponse universelle : chaque décision dépend du contexte et des objectifs.

Décision	Question à poser	Risque si mal décidé
IaaS / PaaS / SaaS Quel niveau de service ?	• Qui administre quoi ? • Quel niveau de contrôle avons-nous ? • Quels sont nos besoins d'exploitation ?	**Responsabilités floues** (ex. sécurité, mises à jour, gestion)
Public / privé / hybride Où placer les données ?	• Quelles données doivent rester sur site ? • Quelles sont les contraintes réglementaires ? • Quel niveau de performance et de latence ?	**Conformité et latence** (ex. RGPD, temps d'accès, souveraineté)
Scalabilité Quel composant grandit ?	• Quels sont les pics de charge attendus ? • Quel composant doit scaler (app, base, réseau) ? • Quel est le modèle de coût (à la demande, réservé) ?	**Coût ou saturation** (ex. surprovisionnement ou manque de capacité)
Conteneurs Quel mode d'exploitation ?	• Avons-nous besoin de portabilité ? • Quel niveau d'orchestration est nécessaire ? • Quelles compétences et cutils disponibles ?	**Complexité inutile** (ex. sur-ingénierie, coûts, maintenance)

À retenir : Une décision Cloud pertinente repose sur l'analyse des besoins métier, des contraintes techniques et des risques, en gardant la valeur au centre.

ÉTUDE DE CAS

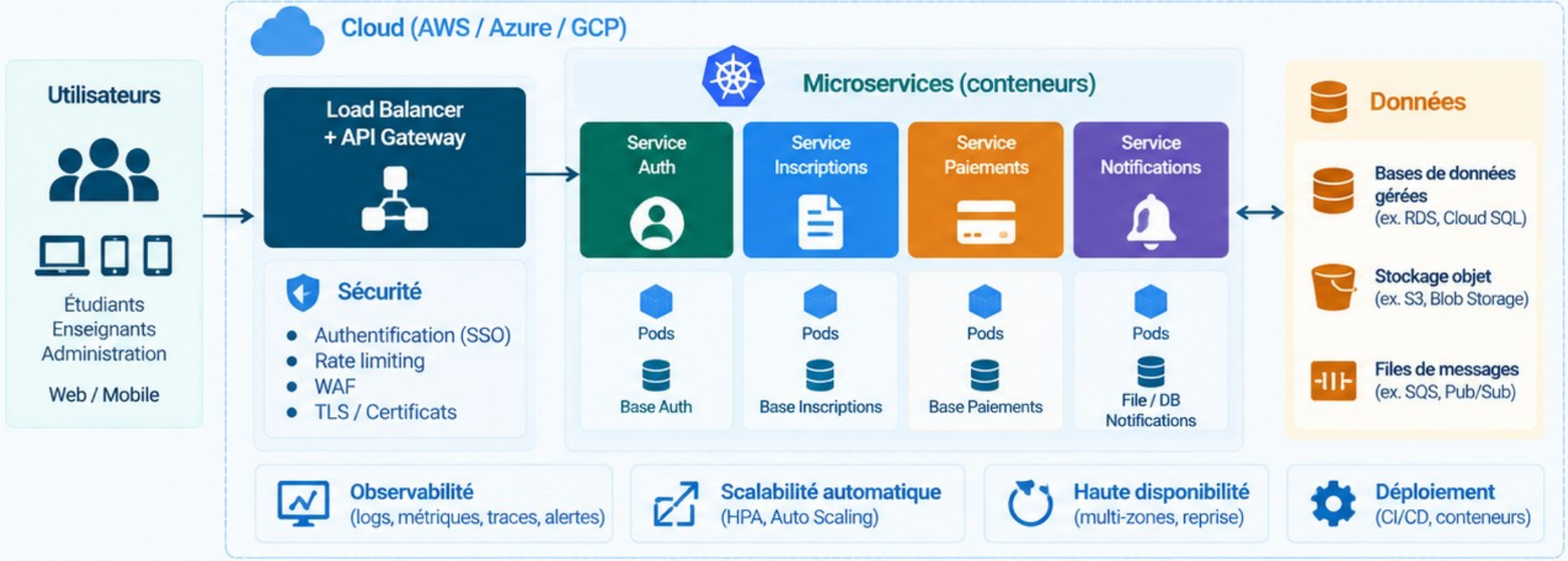
Cas final : microservices dans le Cloud

Une plateforme universitaire moderne, scalable et résiliente

Une plateforme universitaire sépare authentification, inscriptions, paiements et notifications en microservices. Elle doit fonctionner pendant les inscriptions massives et offrir une expérience fiable et sécurisée.

Décisions à justifier

- Choisir REST ou messaging selon les flux
- Positionner API Gateway et load balancer
- Prévoir tolérance aux pannes
- Définir la stratégie de déploiement Cloud



Bénéfices

- Scalabilité lors des inscriptions massives
- Résilience et haute disponibilité
- Évolutivité des services
- Meilleure maintenabilité
- Expérience utilisateur fiable

À retenir : Les microservices dans le Cloud permettent de répondre à des pics de charge, tout en améliorant la résilience, la sécurité et la capacité d'évolution de la plateforme.

SYNTHÈSE

Synthèse des décisions distribué / Cloud

Les bons choix pour un système scalable, résilient et maîtrisé

Une architecture distribuée réussie repose sur des choix alignés avec les besoins métier, les contraintes techniques et les objectifs de l'organisation.

Sujet	Bonne question	Décision attendue
Découpage Architecture et responsabilités	• Quel domaine doit être autonome ? • Comment découper les services ? • Quelles dépendances entre services ?	**Service par responsabilité claire** (ex. auth, inscriptions, paiements, notifications)
Communication Échanges entre services	• Réponse immédiate ou traitement différé ? • Quel protocole utiliser ? • Comment gérer la cohérence des données ?	**REST ou messaging** (ex. HTTP/REST pour synchrone, Kafka/RabbitMQ pour asynchrone)
Cloud Infrastructure et déploiement	• Qui gère l'infrastructure ? • Public, privé ou hybride ? • Quels services managés utiliser ?	**IaaS, PaaS ou hybride** (selon contraintes, sécurité, coûts)
Résilience Continuité de service	• Que faire si un service tombe ? • Quels mécanismes de reprise ? • Comment surveiller et alerter ?	**Timeout, retry, fallback, supervision** (ex. circuit breaker, auto-scaling, alertes)

À retenir : Une architecture distribuée efficace est le résultat de choix cohérents et justifiés, qui équilibrent agilité, performance, sécurité et maîtrise des coûts.